

TFT Screen Library (SPI Version)

Contreras R Orlando^{1,2*} and Lara H Kevin^{1,2*}

³Department of Electronic Systems, UAA, Av. Universidad 940,
Aguascalientes, 20131, Aguascalientes, México.

*Corresponding author(s). E-mail(s): [al348390,al464376}@edu.uaa.mx](mailto:{al348390,al464376}@edu.uaa.mx);

Abstract

The objective of this TFT-LCD screen driver is implement an easy way to show info on a screen. This is very useful in embedded systems applications, like show images, text, or anything that requires a screen.

Keywords: ARM, C++, Driver, TFT Screen.

1 Introduction

Communication protocols are very useful in a lot of tech applications; all computers, micro controllers, and every circuit with a processor need communication protocols to send or receive I/O data and control external peripherals, some of the most used protocols are SPI. Using this application, the ST7735 TFT screen is a device able to print pixels on a point-matrix, sending via SPI the column and row address to set the internal screen pointer on the pixel that we want to turn on, subsequently we send the color data of this pixel in the color format RGB565.

1.1 RGB565

The RGB565 color format is a 16-bit representation of colors, where 5 bits are used for red, 6 bits for green, and 5 bits for blue. This allows for a total of 65,536 different colors. The format is structured as follows:

Here we got the pink color on RGB565 format (0xF81F), the red color (0xF800), the green color (0x07E0), and the blue color (0x001F), while on RGB888 the pink color is (0xFF00FF), the red color (0xFF0000), the green color (0x00FF00), and the blue color (0x0000FF).

Therefore we could get the conversion from RGB888 to RGB565 using the following formula:

$$RGB565 = ((R \& 0xF8) \ll 8) | ((G \& 0xFC) \ll 3) | (B \gg 3) \quad (1)$$

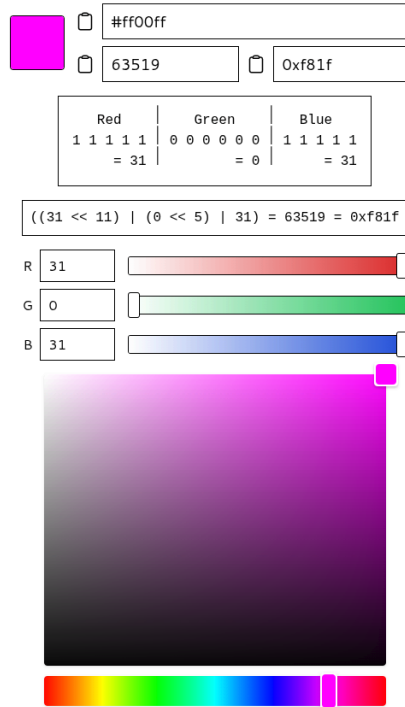


Fig. 1: RGB Format

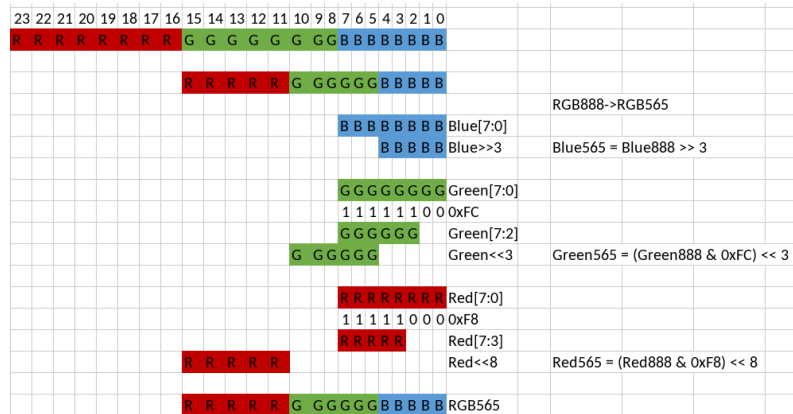


Fig. 2: RGB565 Conversion Formula

If we didnt had the colors itself, we could use the following formula to get the RGB565 color from the RGB888 color:

$$RGB565 = (((RGB888 \& 0xf80000) \gg 8) + ((RGB888 \& 0xfc00) \gg 5) + ((RGB888 \& 0xf8) \gg 3)); \quad (2)$$

2 ST7735

The screen ST7735 handles two interfaces

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
RGB888	0	0	0	0	0	0	0	0	0	r7	r6	r5	r4	r3	r2	r1	r0	g7	g6	g5	g4	g3	g2	g1	g0	b7	b6	b5	b4	b3	b2	b1	b0
mask red	0	0	0	0	0	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
result red	0	0	0	0	0	0	0	0	0	r7	r6	r5	r4	r3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
mask green	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0
result green	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	g7	g6	g5	g4	g3	g2	0	0	0	0	0	0	0	0	0	0
mask blue	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	0	0	0
result blue	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	b7	b6	b5	b4	b3	0	0	0	0
RGB565																	r7	r6	r5	r4	r3	g7	g6	g5	g4	g3	g2	b7	b6	b5	b4	b3	

Fig. 3: RGB Color Conversion Process

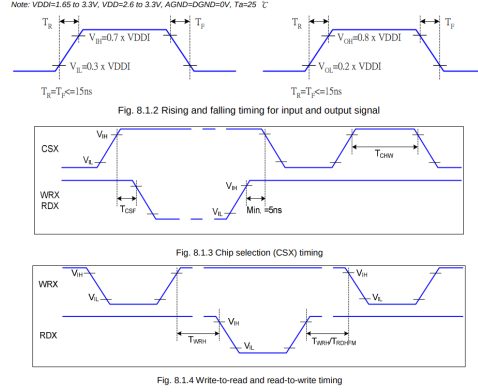


Fig. 4: ST7735 Interface Considerations

- 3-Line/9-bit Serial interface (CS (Chip Select), SCL (Signal Clock), SDA (Serial Data Input/Output))
- 4-Line/8-bit Serial interface (CS (Chip Select), SCL (Signal Clock), SDA (Serial Data Input/Output), D/CX (Data/Command Flag))

We could switch between them with s

$$\begin{cases} SPI4W = 0 & (3\text{-Line Serial interface}) \\ SPI4W = 1 & (4\text{-Line Serial interface}) \end{cases}$$

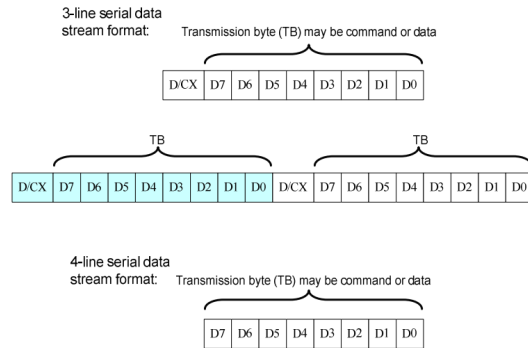


Fig. 5: 3-Line and 4-Line Serial Interface

2.1 Time and Voltage Considerations

Should be considered the following specifications for the ST7735 screen, the voltage levels and the time of the signals, as shown in figure 4. We take the VDDI voltage as 3.3V same as VDD.

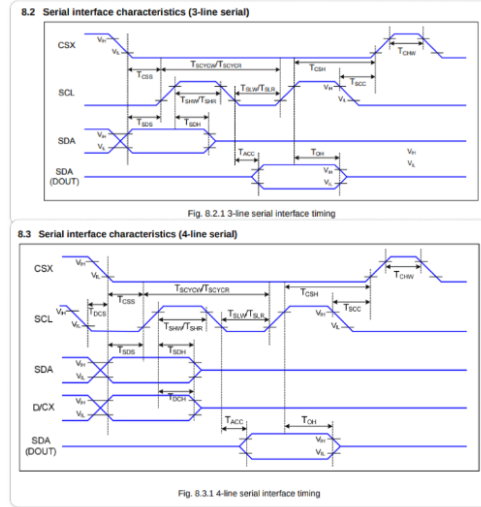


Fig. 6: Timing Constraints

Table 1: 4-line Serial Interface Characteristics

Signal	Symbol	Parameter	MIN	MAX	Unit
CSX	TCSS	Chip select setup time (write)	15		ns
	TCSH	Chip select hold time (write)	15		ns
	TCSS	Chip select setup time (read)	60		ns
	TSCC	Chip select hold time (read)	65		ns
	TCHW	Chip select "H" pulse width	40		ns
SCL	TSCYCW	Serial clock cycle (Write)	66		ns
	TSHW	SCL "H" pulse width (Write)	30		ns
	TSBW	SCL "L" pulse width (Write)	30		ns
	TSCYCR	Serial clock cycle (Read)	150		ns
	TSHR	SCL "H" pulse width (Read)	60		ns
	TSBR	SCL "L" pulse width (Read)	60		ns
D/CX	TDCS	D/CX setup time		0	ns
	TDCH	D/CX hold time	10		ns
SDA (DIN) (DOUT)	TSDS	Data setup time	10		ns
	TSDH	Data hold time	10		ns
	TACC	Access time	10	50	ns
	TOH	Output disable time		50	ns

3 Description

The library for the ST7735-based TFT display provides a simple and efficient interface for handling graphics through the SPI communication protocol.

This library abstracts the complexity of the command sequences and internal operation of the ST7735, offering intuitive functions to initialize the display, print color rectangles, fill the screen with one color, draw pixels, and test the colors on the screen, each one integrated on a 4 simple function to use:

Table 2: Parallel 8080-Series MCU Interface Characteristics

Signal	Symbol	Parameter	MIN	MAX	Unit
D/CX	AST	Address setup time	0		ns
	AHT	Address hold time (Write/Read)	10		ns
CSX	CHW	Chip select “H” pulse width	0		ns
	CSXN	Chip select setup time (Write)	15		ns
	CSRN	Chip select setup time (Read ID)	45		ns
WRX	TWC	Write cycle	66		ns
	WRH	Write Control “H” pulse width	15		ns
	WRL	Write Control “L” pulse width	15		ns
RDX (ID)	TRCFM	Read cycle (FM)	450		ns
	RDHFM	Read Control “H” pulse width (FM)	90		ns
	RDLFM	Read Control “L” pulse width (FM)	355		ns
D[17:0]	DST	Data setup time	10		ns
	DHT	Data hold time	10		ns
	RAT	Read access time (FM)		340	ns
	ODH	Output disable time	20	80	ns

Screen1.FillScreen(color).

Screen1.FillRectangle(posx, posy, width, height, color).

Screen1.DrawPixel(posyx, posy, color). test(void);

3.1 Gamma Adjustments (Positive and Negative Polarity)

The ST7735 controller features highly customizable Gamma Correction registers (ST7735_GMCTRP1 for positive polarity and ST7735_GMCTRN1 for negative polarity). In Liquid Crystal Displays (LCDs), gamma correction is not merely a software tool for contrast styling, but a critical hardware requirement to compensate for the non-linear electro-optical transfer characteristics of the liquid crystal layer.

3.1.1 Understanding Polarity in Gamma Control

Liquid crystal molecules are sensitive to the root-mean-square (RMS) value of an electric field. If a constant DC voltage is applied continuously in a single direction across the liquid crystal cells, it causes ion polarization and chemical degradation, permanently damaging the screen (a phenomenon known as *image sticking* or LCD burn-in).

To prevent this, TFT displays use an Alternating Current (AC) driving method where the polarity of the voltage applied to each pixel is inverted periodically (typically per frame or per line) relative to a common reference voltage (V_{COM}).

- **Positive Polarity (GMCTRP1):** Defines the voltage-to-grayscale mapping curve when the pixel voltage is higher than V_{COM} .
- **Negative Polarity (GMCTRN1):** Defines the voltage-to-grayscale mapping curve when the pixel voltage is lower than V_{COM} .

Because the physical behavior of liquid crystals can vary slightly depending on whether the charge is positive or negative, independent 16-argument register arrays are required to guarantee that a specific grayscale value outputs the exact same optical luminance in both polarities, preventing screen flickering.

3.1.2 Determination of Gamma Values

The 16 hexadecimal parameters passed to each gamma register block represent specific tap points along a digital-to-analog converter (DAC) voltage resistor ladder. These target points adjust key characteristics of the curve:

- **Low-order parameters** alter the black levels and dark gray gradients.
- **Mid-order parameters** fine-tune the mid-tones and linearity of the color distribution.
- **High-order parameters** determine the peak white saturation levels.

In practical driver development, these values are **empirically determined by the display module manufacturer**. The screen vendor utilizes specialized optical instrumentation (such as colorimeters and spectrophotometers) to measure the luminance response across the grayscale spectrum on a bare glass panel sample.

The factory engineers then calculate the exact DAC voltage offsets required to match standard target curves (typically a Gamma 2.2 profile, which matches human visual perception). The resulting magic numbers—such as the ones embedded in our `init_cmd[]` array—are hardcoded into the initialization routine to ensure accurate color reproduction, stable grayscale transitions, and identical luminance across both positive and negative driving cycles.

4 Initialization Sequence

The following array contains the configuration sequence for the ST7735R controller. To distinguish between commands and data control packets within the bare-metal driver, a 0x0100 prefix mask is applied to metadata parameters, while direct commands maintain their hardware definitions.

```

1 uint16_t init_cmd[] = {
2     // Init for 7735R, part 1 (red or green tab)
3     ST7735_SWRESET, // 1: Software reset, 0 args, w/delay
4     ST7735_SLP0UT , // 2: Out of sleep mode, 0 args, w/delay
5     ST7735_FRMCTR1, // 3: Frame rate ctrl - normal mode, 3 args:
6     0x0101,         //      Argument 1: RTNA setting
7     0x012C,         //      Argument 2: FPA setting
8     0x012D,         //      Argument 3: BPA setting. Rate = fosc/(1
9                     x2+40) * (LINE+2C+2D)
10    ST7735_FRMCTR2, // 4: Frame rate control - idle mode, 3 args:
11    0x0101,         //      Argument 1: RTNB setting
12    0x012C,         //      Argument 2: FPB setting
13    0x012D,         //      Argument 3: BPB setting. Rate = fosc/(1
14                     x2+40) * (LINE+2C+2D)
15    ST7735_FRMCTR3, // 5: Frame rate ctrl - partial mode, 6 args:
16    0x0101,         //      Argument 1: RTNC setting
17    0x012C,         //      Argument 2: FPC setting
18    0x012D,         //      Argument 3: BPC setting (Dot inversion
19                     mode)
20    0x0101,         //      Argument 4: RTNC setting
21    0x012C,         //      Argument 5: FPC setting
22    0x012D,         //      Argument 6: BPC setting (Line inversion
23                     mode)
24    ST7735_INVCTR , // 6: Display inversion ctrl, 1 arg, no delay
25    :
26    0x0107,         //      Argument 1: No inversion (Line
27                     inversion on full/idle/partial)
28    ST7735_PWCTR1 , // 7: Power control, 3 args, no delay:
29    0x01A2,         //      Argument 1: GVDD setup
30    0x0102,         //      Argument 2: VCI1 / VGH / VGL setup
31                     (-4.6V)
32    0x0184,         //      Argument 3: Opamp bias selection (AUTO
33                     mode)

```

```

26 ST7735_PWCTR2 , // 8: Power control, 1 arg, no delay:
27 0x01C5, // Argument 1: VGH25 = 2.4C, VGSEL = -10,
    VGH = 3 * AVDD
28 ST7735_PWCTR3 , // 9: Power control, 2 args, no delay:
29 0x010A, // Argument 1: Opamp current small (Normal
    mode)
30 0x0100, // Argument 2: Boost frequency (Normal
    mode)
31 ST7735_PWCTR4 , // 10: Power control, 2 args, no delay:
32 0x018A, // Argument 1: BCLK/2, Opamp current small
    & Medium low (Idle)
33 0x012A, // Argument 2: Boost frequency (Idle mode)
34 ST7735_PWCTR5 , // 11: Power control, 2 args, no delay:
35 0x018A, // Argument 1: Opamp current and boost (
    Partial mode)
36 0x01EE, // Argument 2: Boost frequency (Partial
    mode)
37 ST7735_VMCTR1 , // 12: Power control, 1 arg, no delay:
38 0x010E, // Argument 1: VCOM voltage setting
39 ST7735_INVOFF , // 13: Don't invert display, no args, no
    delay
40 ST7735_MADCTL , // 14: Memory access control (directions), 1
    arg:
41 ST7735_ROTATION, // Argument 1: Row/Col address scan,
    bottom to top refresh
42 ST7735_COLMOD , // 15: Set color mode, 1 arg, no delay:
43 0x0105 , // Argument 1: 16-bit color format (RGB565
    )
44 ST7735_CASET , // 16: Column addr set, 4 args, no delay:
45 0x0100, // Argument 1: XSTART high byte
46 0x0100, // Argument 2: XSTART low byte = 0
47 0x0100, // Argument 3: XEND high byte
48 0x017F, // Argument 4: XEND low byte = 127
49 ST7735_RASET , // 17: Row addr set, 4 args, no delay:
50 0x0100, // Argument 1: YSTART high byte
51 0x0100, // Argument 2: YSTART low byte = 0
52 0x0100, // Argument 3: YEND high byte
53 0x017F, // Argument 4: YEND low byte = 127
54 ST7735_GMCTRP1, // 18: Gamma Adjustments (pos. polarity), 16
    args, no delay:
55 0x0102, 0x011c, 0x0107, 0x0112, // Arguments 1 to 4: Curve
    mapping
56 0x0137, 0x0132, 0x0129, 0x012d, // Arguments 5 to 8: Curve
    mapping
57 0x0129, 0x0125, 0x012B, 0x0139, // Arguments 9 to 12: Curve
    mapping
58 0x0100, 0x0101, 0x0103, 0x0110, // Arguments 13 to 16: Curve
    mapping
59 ST7735_GMCTRN1, // 19: Gamma Adjustments (neg. polarity), 16
    args, no delay:
60 0x0103, 0x011d, 0x0107, 0x0106, // Arguments 1 to 4: Curve
    mapping
61 0x012E, 0x012C, 0x0129, 0x012D, // Arguments 5 to 8: Curve
    mapping
62 0x012E, 0x012E, 0x0137, 0x013F, // Arguments 9 to 12: Curve
    mapping
63 0x0100, 0x0100, 0x0102, 0x0110, // Arguments 13 to 16: Curve
    mapping

```

```

64     ST7735_NORON , // 20: Normal display on, no args, w/delay
65     ST7735_DISPON // 21: Main screen turn on, no args w/delay
66 };

```

5 Purpose

This library aims to support rapid development of visual interfaces, diagnostic screens, and real-time graphical outputs, offering a set of functions optimized for performance and compatibility with a wide range of microcontrollers. It is designed to serve both beginners and advanced users who require a practical and flexible tool to integrate TFT graphics into their projects.

6 How to use properly

This section explains how to integrate and use the ST7735 TFT driver and the oscilloscope demo on an STM32F446 MCU.

6.1 Driver Initialization

1. Instantiate the driver object (constructor calls low-level `config()` for RCC/GPIO/SPI).
2. Call `INIT_FN()` once after reset to send the ST7735 init sequence.
3. Clear or paint the screen using `FillScreen(color)`.

Example:

```

TFT_ST7735 tft;

tft.INIT_FN();

tft.FillScreen(COLOR_BLACK);

```

6.2 Drawing Primitives

- Pixel: `DrawPixel(x, y, color)`
- Rectangle: `FillRectangle(x, y, w, h, color)` (auto clipping)
- Full screen: `FillScreen(color)`
- Text: `WriteChar(x, y, ch, font, fg, bg)` and `WriteString(x, y, str, font, fg, bg)`

Colors use RGB565; predefined constants (`COLOR_RED`, `COLOR_BLUE`, etc.) live in the header.

6.3 Oscilloscope Demo Flow

Provided in `main_osc.cpp`:

1. Configure PLL and system clock via `conf_osc()`.
2. Enable and start continuous conversion on ADC1 channel at PA0.
3. Initialize TFT and draw axes using rectangles.

4. In loop: read ADC1->DR, scale value ($>>5$) to horizontal pixel range, increment vertical time index, plot with `DrawPixel()`.
5. When vertical index reaches bottom, clear plot region and restart for a scrolling effect.

6.4 Build Notes

The code uses bare-metal register access (no HAL). Ensure your linker script and startup file match STM32F446. SPI baud is set to $f_{PCLK}/16$. If the display shows nothing, verify wiring and that HSE/PLL configuration succeeds.

6.5 Testing

1. Power the board and TFT (3.3V only).
2. Flash firmware containing either `main_tst.cpp` (color test) or `main_osc.cpp` (oscilloscope).
3. For oscilloscope: vary the analog input on PA0 (potentiometer or waveform source) and observe horizontal deflection of the cyan trace.

7 Diagram

In figure 7 the test circuit with the test code printing multiple rectangles, notice that the connections are in table 3.

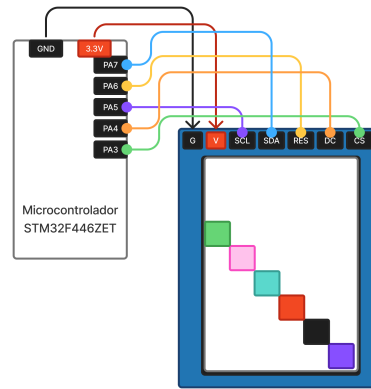


Fig. 7: Schematic

Table 3: TFT to STM32F446 Pin Connections

TFT Pin	STM32F446 Pin	Purpose
VCC	3.3V	Power
GND	GND	Ground
SCK	PA5	SPI1 SCK (AF5)
MOSI	PA7	SPI1 MOSI (AF5)
CS	PA3	Chip Select (GPIO)
DC	PA4	Data/Command select (GPIO)
RST	PA6	Hardware reset (GPIO)

Optional: connect a variable analog signal (e.g. potentiometer wiper) to PA0 for the oscilloscope demo (ADC input).

8 Limitations and Enhancements

8.1 Current Limitations

- **Polling SPI:** All transfers busy-wait; CPU cannot perform other tasks concurrently.
- **No DMA:** Large area fills and waveform plotting are slower than possible; higher CPU usage.
- **Fixed Rotation:** Only one memory access orientation initialized; no runtime rotation switching.
- **Limited Primitives:** Lines, circles, bitmaps, and sprites are not yet implemented.
- **No Text Clipping:** Long strings may draw off-screen without wrapping or truncation.
- **Oscilloscope Scaling:** Simple right-shift scaling ($\gg 5$) reduces resolution; no trigger or persistence.
- **Blocking Clear:** Screen clear for oscilloscope resets full plot; no partial scrolling buffer.

8.2 Potential Enhancements

- **SPI DMA + Interrupts:** Offload pixel bursts; increase frame throughput.
- **Drawing Toolkit:** Add line (Bresenham), circle (Midpoint), bitmap blitting, and gradient fills.
- **Runtime Rotation:** Implement MADCTL updates with offset correction and cached clipping.
- **Text System:** Clipping region, wrapping, proportional fonts, and transparency support.
- **Waveform Features:** Trigger modes (edge/level), adjustable vertical/horizontal scaling, persistence with ring buffer.
- **Double Buffering:** Compose frames off-screen then push for flicker-free updates.
- **UI Layer:** Basic widgets (labels, value bars, status icons) for sensor dashboards.
- **Color/Theme Profiles:** Map amplitude ranges to color gradients (heatmap visualization).

8.3 Recommended Next Steps

Prioritize DMA integration and trigger logic for oscilloscope, then expand primitive set and text handling to support richer UI applications (data logger, multi-channel monitor).

- Configure SPI in master (one board) and slave (other board) mode.
- Configure UART on both boards for terminal output text (Putty etc).
- Define and use two control commands over SPI: start and finish.

- Implement the count logic (0-9) on the slave, and delay/loop logic on the master.
- Print meaningful messages via UART and maintain a cycle counter on the slave